O objetivo é orientar a construção de um sistema web para gerenciamento de serviços automotivos, podendo ser utilizado em uma oficina mecânica ou em um lava-car.

O sistema deverá permitir cadastrar proprietários, registrar seus veículos e acompanhar os serviços solicitados ou realizados.

O projeto também permite comparar duas maneiras de acessar o PostgreSQL:

- **Alternativa A:** utilizar o pacote `pg` para executar comandos SQL diretamente.
- **Alternativa B:** utilizar Sequelize, um ORM que representa as tabelas por meio de Models e oferece métodos para consultar e gravar dados.

As duas alternativas compartilham a mesma modelagem, regras de negócio, autenticação, endpoints HTTP e frontend React. Portanto, não é necessário criar dois frontends: o que muda é a implementação da camada de persistência no backend.

Parte do sistema	Responsabilidade
PostgreSQL	Armazenar usuários, proprietários, veículos e ordens de serviço.
Backend Node.js + Express	Implementar regras de negócio, autenticação e API REST.
Persistência: pg ou Sequelize	Executar consultas e operações no banco de dados.
Frontend React + Axios	Exibir telas, enviar requisições e apresentar os dados.

Ordem recomendada de desenvolvimento

1. Preencher a documentação e compreender os requisitos.
2. Criar o banco e as quatro tabelas no PostgreSQL.
3. Escolher uma estratégia de persistência: pg ou Sequelize.
4. Configurar a conexão e testar a comunicação com o banco.
5. Implementar consultas parametrizadas (pg) ou Models e associações (Sequelize).
6. Implementar cadastro de usuários, bcrypt, login e JWT.
7. Criar controllers e rotas de proprietários, veículos e ordens de serviço.
8. Criar e executar o seed para inserir dados de demonstração.
9. Testar os endpoints com Insomnia ou Postman.
10. Criar o frontend React e configurar a instância Axios.
11. Implementar login, dashboard e componentes de listagem.

Entender o problema e definir requisitos

Uma empresa que presta serviços automotivos precisa organizar os dados dos clientes, seus veículos e os serviços realizados.

Atualmente, os registros podem estar distribuídos em planilhas, anotações ou sistemas diferentes. Isso dificulta localizar o histórico de um veículo, identificar seu proprietário e acompanhar os serviços que ainda estão em andamento.

A proposta é desenvolver uma aplicação web que centralize essas informações.

O mesmo sistema deverá atender a dois cenários:

Cenário 1 — Oficina mecânica

A empresa realiza manutenção preventiva e corretiva, troca de peças, revisão, diagnóstico e outros serviços. Cada atendimento gera uma ordem de serviço que registra o problema relatado, o serviço executado, o valor e o andamento do atendimento.

Cenário 2 — Lava-car

A empresa realiza lavagem simples, lavagem completa, higienização interna, polimento e outros serviços. Cada atendimento registra o serviço contratado, o veículo, o valor e o status da execução.

A estrutura técnica será a mesma. O que muda é a descrição dos serviços utilizados no cadastro.

Requisitos funcionais

ID	Requisito verificável
RF01	Permitir login e informar falha quando as credenciais forem inválidas.
RF02	A sessão autenticada deve expirar após um período definido.
RF03	Exibir o nome do usuário logado, navegação e opção de sair.
RF04	Permitir pesquisar proprietários e atualizar a lista com os resultados.
RF05	Permitir cadastrar veículos vinculados a um proprietário existente.
RF06	Listar veículos apresentando seus respectivos proprietários.
RF07	Permitir cadastrar ordens de serviço vinculadas a um veículo.
RF08	Listar ordens de serviço com dados do veículo e do proprietário, ordenadas pela data.
RF09	Permitir atualizar o status de uma ordem de serviço.
RF10	Proteger dados pessoais e não expor segredos no código-fonte.

RF11 Entregar script SQL com pelo menos três registros por tabela.

RF12 Entregar documentação e DER compatíveis com o banco implementado.

Perguntas que devem ser respondidas antes de começar

- Quais telas serão necessárias e como o usuário navega entre elas?
- Quais campos são obrigatórios em cada cadastro?
- O que deve acontecer se o proprietário informado não existir?
- Como identificar um veículo? Qual informação deve ser única?
- O que acontece quando uma ordem de serviço é concluída?
- Como será comprovado que a busca foi feita pela API e pelo banco?

1. Escopo funcional do sistema

O sistema deverá contemplar:

- Autenticar usuários internos da empresa por e-mail e senha.
- Cadastrar e consultar proprietários.
- Cadastrar e consultar veículos, vinculando cada veículo a um proprietário.
- Cadastrar e consultar ordens de serviço, vinculando cada ordem a um veículo.
- Atualizar o status dos serviços.
- Proteger as rotas da API com token JWT.
- Apresentar um dashboard simples com as listagens principais.

1.1 Entidades e relacionamentos

Entidade	Principais campos	Regra de relacionamento
Usuario	nome, email, senha, perfil	Autentica os funcionários da empresa.
Proprietario	nome, email, telefone, cpf	Um proprietário pode possuir vários veículos.
Veiculo	marca, modelo, placa, ano, cor, proprietarioId	Cada veículo pertence a um proprietário.
OrdemServico	dataEntrada, descricao, tipoServico, valor, status, veiculoId	Cada ordem de serviço pertence a um veículo.

Entendendo cada entidade

Usuario

Representa os funcionários que acessam o sistema.

Exemplos de perfis:

- admin: administra o sistema.
- atendente: realiza cadastros e registra serviços.
- tecnico: executa os serviços e atualiza seu andamento.

Proprietario

Representa o cliente que possui um ou mais veículos.

Campos sugeridos:

- id
- nome
- email
- telefone
- cpf

Veiculo

Representa o automóvel que receberá os serviços.

Campos sugeridos:

- id
- marca
- modelo
- placa
- ano
- cor
- proprietarioId

OrdemServico

Representa um atendimento ou serviço solicitado para determinado veículo.

Campos sugeridos:

- id
- dataEntrada
- descricao
- tipoServico
- valor
- status
- veiculoId

Exemplos de tipos de serviço:

Oficina mecânica	Lava-car
Troca de óleo	Lavagem simples
Revisão preventiva	Lavagem completa
Troca de pastilhas	Higienização interna
Diagnóstico mecânico	Polimento
Alinhamento	Enceramento

Exemplos de status:

- aberto
- em_andamento
- concluido
- cancelado

Regra importante: os tipos de serviço são armazenados como texto. Assim, não é necessário criar tabelas adicionais para cada tipo de serviço.

Relacionamentos

- Proprietario 1 Veiculo.
- Veiculo 1 OrdemServico.

A entidade Usuario não precisa estar ligada diretamente ao proprietário ou veículo no escopo inicial.

2. Organização do projeto

Estrutura sugerida:

```
None
gestao-automotiva/
├── server/
│   ├── src/
│   │   ├── config/
│   │   │   └── database.js
│   │   ├── controllers/
│   │   │   ├── authController.js
│   │   │   ├── proprietarioController.js
│   │   │   ├── veiculoController.js
│   │   │   └── ordemServicoController.js
```

```
|   |   ├── middlewares/
|   |   |   └── auth.js
|   |   ├── models/
|   |   |   ├── Usuario.js
|   |   |   ├── Proprietario.js
|   |   |   ├── Veiculo.js
|   |   |   └── OrdemServico.js
|   |   ├── routes/
|   |   |   └── index.js
|   |   ├── seeders/
|   |   |   └── seed.js
|   |   └── server.js
|   ├── .env
|   └── package.json
└── client/
    ├── .env
    └── src/
        ├── components/
        ├── services/
        |   └── api.js
        ├── App.jsx
        └── App.css
```

A pasta `models` é necessária na alternativa Sequelize. Na alternativa pg, os controllers podem executar as consultas diretamente por meio do pool de conexões.

3. Preparação do ambiente

3.1 Criar o backend

Criar pastas e iniciar projeto via terminal:

Shell

```
mkdir gestao-automotiva
cd gestao-automotiva

mkdir server
cd server
```

```
npm init -y

npm install express cors dotenv pg bcrypt jsonwebtoken
npm install -D nodemon
```

Para utilizar Sequelize, instale também:

```
Shell
npm install sequelize pg-hstore
```

O pacote `pg` já foi instalado no comando principal e também é utilizado pelo Sequelize como driver de comunicação com o PostgreSQL.

3.2 Configurar package.json

No arquivo `server/package.json`, configurar:

```
JSON
{
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js",
    "seed": "node src/seeders/seed.js"
  }
}
```

O atributo `type: module` permite utilizar os comandos `import` e `export` nos arquivos JavaScript no lugar do `require`.

3.3 Criar o frontend

Voltar para a pasta principal:

```
Shell
cd ..
```

```
npm create vite@latest client
```

Selecionar:

- React
- JavaScript
- ESLint

Depois:

```Shell
cd client

npm install
npm install axios
```

3.4 Variáveis de ambiente

Criar o arquivo `server/.env`:

```None
PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_NAME=gestao_automotiva
DB_USER=postgres
DB_PASSWORD=sua_senha

JWT_SECRET=gere_uma_chave_forte

FRONTEND_ORIGIN=http://localhost:5173
```

Criar o arquivo `client/.env`:

```None
VITE_API_URL=http://localhost:3000/api
```

Para criar o banco `gestao_automotiva` no PostgreSQL, podemos utilizar o pgAdmin ou DBeaver.

Gere seus próprios segredos e não publique o arquivo `.env` em repositórios.

4. Estrutura do banco de dados

A estrutura relacional será a mesma nas duas alternativas.

O exemplo SQL cria quatro tabelas, chaves primárias, chaves estrangeiras, restrições de valores e timestamps utilizados nos exemplos Sequelize.

4.1 Criar a tabela de usuários

```sql
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    email VARCHAR(160) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    perfil VARCHAR(20) NOT NULL
        CHECK (perfil IN ('admin', 'atendente', 'tecnico')),
    "createdAt" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

A tabela guarda as informações dos funcionários que possuem acesso ao sistema.

A senha deverá ser armazenada utilizando bcrypt, nunca em texto puro.

4.2 Criar a tabela de proprietários (tabela simples)

```sql
CREATE TABLE proprietarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    email VARCHAR(160) NOT NULL UNIQUE,
    telefone VARCHAR(20) NOT NULL,
    cpf TEXT NOT NULL,
    "createdAt" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);
```

A tabela representa os clientes da oficina ou lava-car.

O CPF será armazenado para identificação, mas não deverá ser exibido em listagens comuns.

4.3 Criar a tabela de veículos (exemplo de tabela com FK da 4.2)

```SQL
CREATE TABLE veiculos (
    id SERIAL PRIMARY KEY,
    marca VARCHAR(80) NOT NULL,
    modelo VARCHAR(100) NOT NULL,
    placa VARCHAR(10) NOT NULL UNIQUE,
    ano INTEGER,
    cor VARCHAR(40),
    "proprietarioId" INTEGER NOT NULL
        REFERENCES proprietarios(id),
    "createdAt" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

A placa identifica o veículo e será única no banco de dados.

A chave estrangeira `proprietarioId` estabelece o relacionamento entre o veículo e seu proprietário.

4.4 Criar a tabela de ordens de serviço

```SQL
CREATE TABLE ordens_servico (
    id SERIAL PRIMARY KEY,
    "dataEntrada" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    descricao VARCHAR(255) NOT NULL,
    "tipoServico" VARCHAR(100) NOT NULL,
    valor NUMERIC(10,2) NOT NULL DEFAULT 0
        CHECK (valor >= 0),
```

```
    status VARCHAR(20) NOT NULL DEFAULT 'aberto'
        CHECK (
            status IN (
                'aberto',
                'em_andamento',
                'concluido',
                'cancelado'
            )
        ),
    "veiculoId" INTEGER NOT NULL
        REFERENCES veiculos(id),
    "createdAt" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

A ordem de serviço registra o atendimento prestado ao veículo.

O campo `descricao` permite explicar o que o cliente solicitou ou qual problema foi identificado.

O campo `tipoServico` identifica a atividade executada.

O campo `valor` representa o valor cobrado pelo atendimento.

O campo `status` permite acompanhar o andamento do serviço.

Podemos substituir a validação da tabela ordens_serviço por um ENUM

```
None
CREATE TYPE status_ordem AS ENUM (

    'aberto',

    'em_andamento',

    'concluido',

    'cancelado'

);
```

e

```
None
CREATE TABLE ordens_servico (
    id SERIAL PRIMARY KEY,
    "dataEntrada" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    descricao VARCHAR(255) NOT NULL,
    "tipoServico" VARCHAR(100) NOT NULL,
    valor NUMERIC(10,2) NOT NULL DEFAULT 0
        CHECK (valor >= 0),
    status status_ordem NOT NULL DEFAULT 'aberto',
    "veiculoId" INTEGER NOT NULL
        REFERENCES veiculos(id),
    "createdAt" TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT NOW()

);
```

4.5 Entendendo o DER

O banco possui dois relacionamentos principais:

```
None
PROPRIETARIOS
      |
      | 1:N
      |
   VEICULOS
      |
      | 1:N
      |
ORDENS_SERVICO

USUARIOS
(autenticação independente)
```

A relação 1 significa que:

- Um proprietário pode ter vários veículos.
- Um veículo pertence a apenas um proprietário.

- Um veículo pode possuir várias ordens de serviço.
- Uma ordem de serviço pertence a apenas um veículo.

A chave estrangeira é responsável por manter essa relação.

5. Alternativa A — SQL com pg (sem Sequelize)

Nesta opção, utilizamos instruções SQL diretamente.

O pacote `pg` envia as consultas ao PostgreSQL. Os resultados de SELECT ficam disponíveis em `result.rows`.

Não são necessários Models.

5.1 Conexão com PostgreSQL

Arquivo `src/config/database.js`:

```javascript
import pg from "pg";
import dotenv from "dotenv";

dotenv.config();

const { Pool } = pg;

export const pool = new Pool({
    host: process.env.DB_HOST,
    port: Number(process.env.DB_PORT || 5432),
    database: process.env.DB_NAME,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD
});
```

O Pool mantém um conjunto de conexões para reutilização. Isso evita abrir uma nova conexão física com o banco a cada requisição.

5.2 Listar e cadastrar proprietários

Arquivo `src/controllers/proprietarioController.js`:

JavaScript

```javascript
import { pool } from "../config/database.js";

export const listarProprietarios = async (req, res) => {
    try {
        const resultado = await pool.query(`
            SELECT id, nome, email, telefone
            FROM proprietarios
            ORDER BY nome
        `);

        return res.json(resultado.rows);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar proprietários."
        });
    }
};

export const criarProprietario = async (req, res) => {
    try {
        const { nome, email, telefone, cpf } = req.body;

        const resultado = await pool.query(
            `INSERT INTO proprietarios
                (nome, email, telefone, cpf)
             VALUES ($1, $2, $3, $4)
             RETURNING id, nome, email, telefone`,
            [nome, email, telefone, cpf]
        );

        return res.status(201).json(resultado.rows[0]);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível cadastrar proprietário."
        });
    }
};
```

Os marcadores `$1`, `$2`, `$3` e `$4` são parâmetros da consulta.

Os valores são enviados separadamente, evitando concatenar texto recebido do usuário diretamente no SQL e reduzindo o risco de SQL Injection.

5.3 Listar veículos e dados do proprietário com JOIN

Arquivo `src/controllers/veiculoController.js`:

JavaScript

```
import { pool } from "../config/database.js";

export const listarVeiculos = async (req, res) => {
    try {
        const resultado = await pool.query(`
            SELECT
                v.id,
                v.marca,
                v.modelo,
                v.placa,
                v.ano,
                v.cor,
                p.id AS "proprietarioId",
                p.nome AS "proprietarioNome"
            FROM veiculos v
            INNER JOIN proprietarios p
                ON p.id = v."proprietarioId"
            ORDER BY v.modelo
        `);

        return res.json(resultado.rows);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar veículos."
        });
    }
};
```

O `INNER JOIN` combina as linhas de veículos e proprietários utilizando a chave estrangeira `proprietarioId`.

Assim, a API pode retornar o nome do proprietário junto dos dados do veículo.

5.4 Cadastrar um veículo

Ainda no `veiculoController.js`:

JavaScript

```
export const criarVeiculo = async (req, res) => {
    try {
        const {
            marca,
            modelo,
            placa,
            ano,
            cor,
            proprietarioId
        } = req.body;

        const resultado = await pool.query(
            `INSERT INTO veiculos
                (marca, modelo, placa, ano, cor,
"proprietarioId")
             VALUES ($1, $2, $3, $4, $5, $6)
             RETURNING *`,
            [marca, modelo, placa, ano, cor, proprietarioId]
        );

        return res.status(201).json(resultado.rows[0]);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível cadastrar veículo."
        });
    }
};
```

A chave estrangeira impede cadastrar um veículo vinculado a um proprietário inexistente.

A placa também deve respeitar a restrição de unicidade.

5.5 Criar ordem de serviço

Arquivo `src/controllers/ordemServicoController.js`:

```javascript
import { pool } from "../config/database.js";

export const criarOrdemServico = async (req, res) => {
    try {
        const {
            descricao,
            tipoServico,
            valor,
            veiculoId
        } = req.body;

        const resultado = await pool.query(
            `INSERT INTO ordens_servico
                (descricao, "tipoServico", valor, "veiculoId")
             VALUES ($1, $2, $3, $4)
             RETURNING *`,
            [descricao, tipoServico, valor, veiculoId]
        );

        return res.status(201).json(resultado.rows[0]);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível criar ordem de serviço."
        });
    }
};
```

O status será preenchido automaticamente com `aberto`, conforme definido no banco de dados.

5.6 Listar ordens de serviço com JOIN

```javascript
export const listarOrdensServico = async (req, res) => {
    try {
        const resultado = await pool.query(`
```

```
            SELECT
                os.id,
                os."dataEntrada",
                os.descricao,
                os."tipoServico",
                os.valor,
                os.status,
                v.placa,
                v.modelo,
                v.marca,
                p.nome AS "proprietarioNome"
            FROM ordens_servico os
            INNER JOIN veiculos v
                ON v.id = os."veiculoId"
            INNER JOIN proprietarios p
                ON p.id = v."proprietarioId"
            ORDER BY os."dataEntrada" DESC
        `);

        return res.json(resultado.rows);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar ordens de serviço."
        });
    }
};
```

Observe que agora utilizamos dois `INNER JOIN`.

A primeira relação localiza o veículo da ordem de serviço.

A segunda relação localiza o proprietário daquele veículo.

Com isso, a listagem apresenta todas as informações necessárias sem precisar fazer várias requisições diferentes.

5.7 Atualizar o status da ordem de serviço

JavaScript

```
export const atualizarStatus = async (req, res) => {
    try {
        const { id } = req.params;
        const { status } = req.body;

        const resultado = await pool.query(
            `UPDATE ordens_servico
             SET status = $1, "updatedAt" = NOW()
             WHERE id = $2
             RETURNING *`,
            [status, id]
        );

        if (resultado.rowCount === 0) {
            return res.status(404).json({
                mensagem: "Ordem de serviço não encontrada."
            });
        }

        return res.json(resultado.rows[0]);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível atualizar o status."
        });
    }
};
```

O exemplo utiliza a restrição de valores da coluna status. Uma atualização com valor diferente dos permitidos será rejeitada pelo PostgreSQL.

6. Alternativa B — Sequelize

Sequelize é um ORM (Object-Relational Mapper).

O desenvolvedor descreve cada tabela por meio de um Model JavaScript.

Métodos como:

- `findAll()`

- `findOne()`
- `create()`
- `update()`
- `findOrCreate()`

representam operações de consulta e persistência.

As associações entre Models permitem carregar dados relacionados.

6.1 Conexão

Arquivo `src/config/database.js`:

```javascript
import { Sequelize } from "sequelize";
import dotenv from "dotenv";

dotenv.config();

export const sequelize = new Sequelize(
    process.env.DB_NAME,
    process.env.DB_USER,
    process.env.DB_PASSWORD,
    {
        host: process.env.DB_HOST,
        port: Number(process.env.DB_PORT || 5432),
        dialect: "postgres",
        logging: false
    }
);
```

6.2 Model Proprietario

Arquivo `src/models/Proprietario.js`:

```javascript
import { DataTypes } from "sequelize";
import { sequelize } from "../config/database.js";

export const Proprietario = sequelize.define(
```

```
    "Proprietario",
    {
        nome: {
            type: DataTypes.STRING(120),
            allowNull: false
        },
        email: {
            type: DataTypes.STRING(160),
            allowNull: false,
            unique: true
        },
        telefone: {
            type: DataTypes.STRING(20),
            allowNull: false
        },
        cpf: {
            type: DataTypes.TEXT,
            allowNull: false
        }
    },
    {
        tableName: "proprietarios",
        timestamps: true
    }
);
```

O Model representa a tabela `proprietarios`.

Os atributos declarados correspondem às colunas do banco de dados.

6.3 Model Veiculo

Arquivo `src/models/Veiculo.js`:

JavaScript
```
import { DataTypes } from "sequelize";
import { sequelize } from "../config/database.js";

export const Veiculo = sequelize.define(
```

```
    "Veiculo",
    {
        marca: {
            type: DataTypes.STRING(80),
            allowNull: false
        },
        modelo: {
            type: DataTypes.STRING(100),
            allowNull: false
        },
        placa: {
            type: DataTypes.STRING(10),
            allowNull: false,
            unique: true
        },
        ano: {
            type: DataTypes.INTEGER
        },
        cor: {
            type: DataTypes.STRING(40)
        }
    },
    {
        tableName: "veiculos",
        timestamps: true
    }
);
```

O campo `proprietarioId` será adicionado pela associação entre os Models.

6.4 Model OrdemServico

Arquivo `src/models/OrdemServico.js`:

JavaScript
```
import { DataTypes } from "sequelize";
import { sequelize } from "../config/database.js";

export const OrdemServico = sequelize.define(
```

```
    "OrdemServico",
    {
        dataEntrada: {
            type: DataTypes.DATE,
            allowNull: false,
            defaultValue: DataTypes.NOW
        },
        descricao: {
            type: DataTypes.STRING(255),
            allowNull: false
        },
        tipoServico: {
            type: DataTypes.STRING(100),
            allowNull: false
        },
        valor: {
            type: DataTypes.DECIMAL(10, 2),
            allowNull: false,
            defaultValue: 0,
            validate: {
                min: 0
            }
        },
        status: {
            type: DataTypes.STRING(20),
            allowNull: false,
            defaultValue: "aberto",
            validate: {
                isIn: [[
                    "aberto",
                    "em_andamento",
                    "concluido",
                    "cancelado"
                ]]
            }
        }
    },
    {
        tableName: "ordens_servico",
```

```
        timestamps: true
    }
);
```

O Model define as regras de validação utilizadas pelo Sequelize. As restrições do banco continuam sendo importantes para proteger os dados.

aqui o status tambem pode ser alterado por um enum

```
None
status: {
    type: DataTypes.ENUM(
        "aberto",
        "em_andamento",
        "concluido",
        "cancelado"
    ),
    allowNull: false,
    defaultValue: "aberto"
}
```

6.5 Model Usuario

Arquivo `src/models/Usuario.js`:

```
JavaScript
import { DataTypes } from "sequelize";
import { sequelize } from "../config/database.js";

export const Usuario = sequelize.define(
```

```
    "Usuario",
    {
        nome: {
            type: DataTypes.STRING(120),
            allowNull: false
        },
        email: {
            type: DataTypes.STRING(160),
            allowNull: false,
            unique: true
        },
        senha: {
            type: DataTypes.STRING(255),
            allowNull: false
        },
        perfil: {
            type: DataTypes.STRING(20),
            allowNull: false,
            validate: {
                isIn: [[
                    "admin",
                    "atendente",
                    "tecnico"
                ]]
            }
        }
    },
    {
        tableName: "usuarios",
        timestamps: true
    }
);
```

6.6 Associações entre Models

Arquivo `src/models/index.js`:

JavaScript

```
import { Usuario } from "./Usuario.js";
import { Proprietario } from "./Proprietario.js";
import { Veiculo } from "./Veiculo.js";
import { OrdemServico } from "./OrdemServico.js";

// Proprietário possui vários veículos
Proprietario.hasMany(Veiculo, {
    foreignKey: "proprietarioId",
    as: "veiculos"
});

Veiculo.belongsTo(Proprietario, {
    foreignKey: "proprietarioId",
    as: "proprietario"
});

// Veículo possui várias ordens de serviço
Veiculo.hasMany(OrdemServico, {
    foreignKey: "veiculoId",
    as: "ordensServico"
});

OrdemServico.belongsTo(Veiculo, {
    foreignKey: "veiculoId",
    as: "veiculo"
});

export {
    Usuario,
    Proprietario,
    Veiculo,
    OrdemServico
};
```

O método `hasMany()` representa uma relação de um para muitos.

O método `belongsTo()` representa a relação de pertencimento.

O alias (apelido) informado em `include`, por exemplo `as: "proprietario"`, deve ser igual ao alias definido na associação.

6.7 Controllers com Sequelize

Listar e cadastrar proprietários

Arquivo `src/controllers/proprietarioController.js`:

```javascript
import { Proprietario } from "../models/index.js";

export const listarProprietarios = async (req, res) => {
    try {
        const proprietarios = await Proprietario.findAll({
            attributes: {
                exclude: ["cpf"]
            },
            order: [["nome", "ASC"]]
        });

        return res.json(proprietarios);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar proprietários."
        });
    }
};

export const criarProprietario = async (req, res) => {
    try {
        const {
            nome,
            email,
            telefone,
            cpf
        } = req.body;

        const proprietario = await Proprietario.create({
            nome,
            email,
            telefone,
            cpf
```

```
        });

        return res.status(201).json({
            id: proprietario.id,
            nome: proprietario.nome,
            email: proprietario.email,
            telefone: proprietario.telefone
        });

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível cadastrar proprietário."
        });
    }
};
```

O método `findAll()` retorna todos os registros.

O método `create()` cria um novo registro.

O atributo `exclude` evita que o CPF seja retornado nessa listagem.

Listar veículos com proprietário

Arquivo `src/controllers/veiculoController.js`:

JavaScript

```
import {
    Veiculo,
    Proprietario
} from "../models/index.js";

export const listarVeiculos = async (req, res) => {
    try {
        const veiculos = await Veiculo.findAll({
            include: [{
                model: Proprietario,
                as: "proprietario",
                attributes: ["id", "nome"]
            }],
```

```
            order: [["modelo", "ASC"]]
        });

        return res.json(veiculos);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar veículos."
        });
    }
};
```

Com o Sequelize, utilizamos `include` para carregar os dados relacionados.

A mesma operação escrita com pg exigiria um SELECT com JOIN.

Cadastrar veículo

JavaScript

```
export const criarVeiculo = async (req, res) => {
    try {
        const {
            marca,
            modelo,
            placa,
            ano,
            cor,
            proprietarioId
        } = req.body;

        const veiculo = await Veiculo.create({
            marca,
            modelo,
            placa,
            ano,
            cor,
            proprietarioId
        });

        return res.status(201).json(veiculo);
```

```
    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível cadastrar veículo."
        });
    }
};
```

Criar ordem de serviço

Arquivo `src/controllers/ordemServicoController.js`:

JavaScript

```
import {
    OrdemServico,
    Veiculo
} from "../models/index.js";

export const criarOrdemServico = async (req, res) => {
    try {
        const {
            descricao,
            tipoServico,
            valor,
            veiculoId
        } = req.body;

        const ordem = await OrdemServico.create({
            descricao,
            tipoServico,
            valor,
            veiculoId
        });

        return res.status(201).json(ordem);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível criar ordem de serviço."
```

```
        });
    }
};
```

Listar ordens de serviço com veículo e proprietário

JavaScript

```
import {
    OrdemServico,
    Veiculo,
    Proprietario
} from "../models/index.js";

export const listarOrdensServico = async (req, res) => {
    try {
        const ordens = await OrdemServico.findAll({
            include: [{
                model: Veiculo,
                as: "veiculo",
                include: [{
                    model: Proprietario,
                    as: "proprietario",
                    attributes: ["id", "nome"]
                }]
            }],
            order: [["dataEntrada", "DESC"]]
        });

        return res.json(ordens);

    } catch (erro) {
        return res.status(500).json({
            mensagem: "Erro ao listar ordens de serviço."
        });
    }
};
```

O Sequelize utiliza as associações para carregar os dados do veículo e do proprietário.

Atualizar status

JavaScript

```
export const atualizarStatus = async (req, res) => {
    try {
        const { id } = req.params;
        const { status } = req.body;

        const ordem = await OrdemServico.findByPk(id);

        if (!ordem) {
            return res.status(404).json({
                mensagem: "Ordem de serviço não encontrada."
            });
        }

        await ordem.update({ status });

        return res.json(ordem);

    } catch (erro) {
        return res.status(400).json({
            mensagem: "Não foi possível atualizar o status."
        });
    }
};
```

6.8 Criar as tabelas com Sequelize

Depois de importar os Models e registrar as associações:

JavaScript

```
import { sequelize } from "./config/database.js";
import "./models/index.js";

await sequelize.authenticate();

await sequelize.sync();
```

O método `authenticate()` verifica a conexão com o banco.

O método `sync()` sincroniza os Models com as tabelas.

Atenção: utilize `sync()` apenas no ambiente de desenvolvimento e com o banco preparado para isso. Não utilize `sync({ alter: true })` ou `sync({ force: true })` indiscriminadamente em bancos que já possuem dados. Para ambientes reais, prefira migrations.

7. Autenticação, rotas e servidor

A autenticação será igual nas duas alternativas.

O usuário informa e-mail e senha. O backend localiza o usuário, compara a senha utilizando bcrypt e gera um token JWT.

7.1 Middleware JWT

Arquivo `src/middlewares/auth.js`:

JavaScript

```
import jwt from "jsonwebtoken";

export const autenticar = (req, res, next) => {
    const token = req.headers.authorization?.split(" ")[1];

    if (!token) {
        return res.status(401).json({
            mensagem: "Token não informado."
        });
    }

    try {
        req.usuario = jwt.verify(
            token,
            process.env.JWT_SECRET
        );

        return next();

    } catch {
        return res.status(401).json({
            mensagem: "Token inválido ou expirado."
        });
```

```
    }
};
```

O middleware verifica se existe um token válido.

Caso esteja válido, permite que a requisição continue.

Caso não esteja válido, retorna HTTP 401.

7.2 Login

A comparação da senha e a criação do JWT são iguais nas duas versões.

O que muda é a forma de localizar o usuário.

Consulta com pg

```JavaScript
const resultado = await pool.query(
    `SELECT id, nome, email, senha, perfil
     FROM usuarios
     WHERE email = $1`,
    [email]
);

const usuario = resultado.rows[0];
```

Consulta com Sequelize

```JavaScript
const usuario = await Usuario.findOne({
    where: { email }
});
```

Lógica comum às duas alternativas

```JavaScript
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
```

```
const senhaValida = usuario &&
    await bcrypt.compare(senha, usuario.senha);

if (!senhaValida) {
    return res.status(401).json({
        mensagem: "Credenciais inválidas."
    });
}

const token = jwt.sign(
    {
        id: usuario.id,
        perfil: usuario.perfil
    },
    process.env.JWT_SECRET,
    {
        expiresIn: "30m"
    }
);

return res.json({
    token,
    usuario: {
        id: usuario.id,
        nome: usuario.nome,
        perfil: usuario.perfil
    }
});
```

O exemplo pressupõe que o usuário já esteja cadastrado com senha gerada por bcrypt.

7.3 Rotas Express

Arquivo `src/routes/index.js`:

JavaScript

```
import { Router } from "express";

import { autenticar } from "../middlewares/auth.js";
```

```
import { login } from "../controllers/authController.js";

import {
    listarProprietarios,
    criarProprietario
} from "../controllers/proprietarioController.js";

import {
    listarVeiculos,
    criarVeiculo
} from "../controllers/veiculoController.js";

import {
    listarOrdensServico,
    criarOrdemServico,
    atualizarStatus
} from "../controllers/ordemServicoController.js";

const router = Router();

// Autenticação
router.post("/auth/login", login);

// Proprietários
router.get("/proprietarios", autenticar, listarProprietarios);
router.post("/proprietarios", autenticar, criarProprietario);

// Veículos
router.get("/veiculos", autenticar, listarVeiculos);
router.post("/veiculos", autenticar, criarVeiculo);

// Ordens de serviço
router.get("/ordens-servico", autenticar, listarOrdensServico);
router.post("/ordens-servico", autenticar, criarOrdemServico);

router.patch(
    "/ordens-servico/:id/status",
    autenticar,
```

```
    atualizarStatus
);

export default router;
```

Para completar o CRUD, crie também as rotas PUT/PATCH e DELETE necessárias e implemente os controllers correspondentes.

Proteja as rotas de acordo com os perfis autorizados.

7.4 Inicialização do servidor

Arquivo `src/server.js`:

JavaScript

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";

import router from "./routes/index.js";

dotenv.config();

const app = express();

app.use(cors({
    origin: process.env.FRONTEND_ORIGIN
}));

app.use(express.json());

app.use("/api", router);

const iniciar = async () => {
    try {
        // Opção pg:
        // importe pool e execute await pool.query("SELECT 1").

        // Opção Sequelize:
```

```
        // importe sequelize e os Models/associações;
        // execute await sequelize.authenticate().

        app.listen(process.env.PORT || 3000, () => {
            console.log("Servidor iniciado.");
        });

    } catch (erro) {
        console.error("Falha ao iniciar:", erro);
        process.exit(1);
    }
};

iniciar();
```

Na opção pg, teste a conexão com `pool.query("SELECT 1")`.

Na opção Sequelize, teste com `sequelize.authenticate()`.

O servidor deverá iniciar somente depois de confirmar a conexão com o banco.

8. Seed — Popular o banco

A carga inicial cria registros de demonstração para testar relacionamentos e telas.

O objetivo é ter, no mínimo:

- Três usuários.
- Três proprietários.
- Três veículos.
- Três ordens de serviço.

Os registros dependentes devem utilizar os IDs gerados pelas tabelas anteriores.

8.1 Seed com SQL/pg

Na alternativa sem Sequelize, o seed pode ser executado como arquivo `.sql` com INSERTs ou como script JavaScript que utiliza `pool.query()`.

Para usuários, gere o hash da senha com bcrypt antes do INSERT.

Exemplo:

JavaScript

```
const proprietario = await pool.query(
    `INSERT INTO proprietarios
        (nome, email, telefone, cpf)
     VALUES ($1, $2, $3, $4)
     RETURNING id`,
    [
        "Carlos Silva",
        "carlos@email.com",
        "48999990000",
        "11122233344"
    ]
);

const proprietarioId = proprietario.rows[0].id;

const veiculo = await pool.query(
    `INSERT INTO veiculos
        (marca, modelo, placa, ano, cor, "proprietarioId")
     VALUES ($1, $2, $3, $4, $5, $6)
     RETURNING id`,
    [
        "Volkswagen",
        "Gol",
        "ABC1D23",
        2020,
        "Branco",
        proprietarioId
    ]
);

const veiculoId = veiculo.rows[0].id;

await pool.query(
    `INSERT INTO ordens_servico
        (descricao, "tipoServico", valor, "veiculoId")
     VALUES ($1, $2, $3, $4)`,
    [
        "Manutenção preventiva",
        "Troca de óleo",
```

```
        180.00,
        veiculoId
    ]
);
```

Repita a operação para os três proprietários, veículos e ordens de serviço.

Utilize uma transação e condições de unicidade para evitar deixar registros parciais ou duplicados em reexecuções.

8.2 Seed com Sequelize

Na alternativa Sequelize, o script importa os Models e utiliza `findOrCreate()` para localizar ou criar registros.

Exemplo de proprietário:

JavaScript
```
const [proprietario] = await Proprietario.findOrCreate({
    where: {
        email: "carlos@email.com"
    },
    defaults: {
        nome: "Carlos Silva",
        email: "carlos@email.com",
        telefone: "48999990000",
        cpf: "11122233344"
    },
    transaction
});
```

Exemplo de veículo:

JavaScript
```
const [veiculo] = await Veiculo.findOrCreate({
    where: {
        placa: "ABC1D23"
    },
    defaults: {
        marca: "Volkswagen",
```

```
        modelo: "Gol",
        placa: "ABC1D23",
        ano: 2020,
        cor: "Branco",
        proprietarioId: proprietario.id
    },
    transaction
});
```

Exemplo de ordem de serviço:

JavaScript
```
await OrdemServico.findOrCreate({
    where: {
        descricao: "Manutenção preventiva",
        veiculoId: veiculo.id
    },
    defaults: {
        descricao: "Manutenção preventiva",
        tipoServico: "Troca de óleo",
        valor: 180.00,
        status: "aberto",
        veiculoId: veiculo.id
    },
    transaction
});
```

O script completo deverá criar também os três usuários com senhas criptografadas por bcrypt.

A operação deverá ser executada dentro de uma transação, com `commit` ao finalizar ou `rollback` caso ocorra algum erro.

Exemplo de execução:

Shell
```
npm run seed
```

9. Frontend React com Axios

A interface pode ser igual nas duas alternativas porque o navegador conversa com a API HTTP, não diretamente com o PostgreSQL.

Para manter o frontend independente do mecanismo de persistência, preserve os caminhos e o formato JSON das respostas.

9.1 Configurar Axios

Arquivo `client/src/services/api.js`:

```javascript
import axios from "axios";

const api = axios.create({
    baseURL: import.meta.env.VITE_API_URL
});

api.interceptors.request.use((config) => {
    const token = sessionStorage.getItem("token");

    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }

    return config;
});

export default api;
```

O Axios cria uma instância centralizada para as requisições.

O interceptor adiciona o token JWT automaticamente no cabeçalho Authorization.

9.2 Login com arrow function e async/await

Arquivo `client/src/components/Login.jsx`:

```
import { useState } from "react";
```

```
import api from "../services/api";

const Login = ({ onLogin }) => {
    const [email, setEmail] = useState("");
    const [senha, setSenha] = useState("");
    const [erro, setErro] = useState("");

    const realizarLogin = async (event) => {
        event.preventDefault();

        setErro("");

        try {
            const resposta = await api.post(
                "/auth/login",
                { email, senha }
            );

            sessionStorage.setItem(
                "token",
                resposta.data.token
            );

            onLogin(resposta.data.usuario);

        } catch (error) {
            setErro(
                error.response?.data?.mensagem ||
                "Falha ao entrar."
            );
        }
    };

    return (
        <form onSubmit={realizarLogin}>
            <h2>Gestão Automotiva</h2>

            <label htmlFor="email">E-mail</label>
```

```
            <input
                id="email"
                type="email"
                value={email}
                onChange={(event) =>
                    setEmail(event.target.value)
                }
                required
            />

            <label htmlFor="senha">Senha</label>

            <input
                id="senha"
                type="password"
                value={senha}
                onChange={(event) =>
                    setSenha(event.target.value)
                }
                required
            />

            <button type="submit">
                Entrar
            </button>

            {erro && <p>{erro}</p>}
        </form>
    );
};

export default Login;
```

O método `preventDefault()` evita que o formulário recarregue a página.

O `api.post()` envia os dados ao backend.

O `await` aguarda a resposta da API antes de continuar a execução.

9.3 Listar proprietários usando useEffect

Arquivo `client/src/components/Proprietarios.jsx`:

```
None
import { useEffect, useState } from "react";
import api from "../services/api";

const Proprietarios = () => {
    const [proprietarios, setProprietarios] = useState([]);
    const [carregando, setCarregando] = useState(true);
    const [erro, setErro] = useState("");

    const carregarProprietarios = async () => {
        try {
            setCarregando(true);

            const resposta = await api.get("/proprietarios");

            setProprietarios(resposta.data);

        } catch (error) {
            setErro("Não foi possível carregar proprietários.");

        } finally {
            setCarregando(false);
        }
    };

    useEffect(() => {
        carregarProprietarios();
    }, []);

    if (carregando) {
        return <p>Carregando...</p>;
    }

    if (erro) {
        return <p>{erro}</p>;
    }

    return (
        <section>
```

```
            <h2>Proprietários</h2>

            {proprietarios.map((proprietario) => (
                <p key={proprietario.id}>
                    {proprietario.nome} -
                    {" "}{proprietario.email}
                </p>
            ))}
        </section>
    );
};

export default Proprietarios;
```

O callback de `useEffect` não é async. Ele chama uma função assíncrona definida no componente.

Dessa forma, o efeito não retorna uma Promise no lugar da função de limpeza opcional do React.

9.4 Listar veículos

Arquivo `client/src/components/Veiculos.jsx`:

```
None
import { useEffect, useState } from "react";
import api from "../services/api";

const Veiculos = () => {
    const [veiculos, setVeiculos] = useState([]);
    const [carregando, setCarregando] = useState(true);
    const [erro, setErro] = useState("");

    const carregarVeiculos = async () => {
        try {
            const resposta = await api.get("/veiculos");

            setVeiculos(resposta.data);
```

```
        } catch (error) {
            setErro("Erro ao carregar veículos.");

        } finally {
            setCarregando(false);
        }
    };

    useEffect(() => {
        carregarVeiculos();
    }, []);

    if (carregando) return <p>Carregando...</p>;

    if (erro) return <p>{erro}</p>;

    return (
        <section>
            <h2>Veículos</h2>

            {veiculos.map((veiculo) => (
                <div key={veiculo.id}>
                    <p>
                        {veiculo.marca} {veiculo.modelo}
                    </p>

                    <p>Placa: {veiculo.placa}</p>

                    <p>
                        Proprietário: {
                            veiculo.proprietario?.nome ||
                            veiculo.proprietarioNome
                        }
                    </p>
                </div>
            ))}
        </section>
    );
};
```

```
export default Veiculos;
```

A propriedade do proprietário pode variar conforme o formato da resposta da API.

No exemplo pg, o nome vem em `proprietarioNome`.

No exemplo Sequelize, o nome vem dentro do objeto `proprietario`.

Para uma aplicação final, recomenda-se padronizar o JSON das duas alternativas.

9.5 Listar ordens de serviço

Arquivo `client/src/components/OrdensServico.jsx`:

```
None
import { useEffect, useState } from "react";
import api from "../services/api";

const OrdensServico = () => {
    const [ordens, setOrdens] = useState([]);
    const [carregando, setCarregando] = useState(true);
    const [erro, setErro] = useState("");

    const carregarOrdens = async () => {
        try {
            const resposta = await api.get("/ordens-servico");

            setOrdens(resposta.data);

        } catch (error) {
            setErro("Erro ao carregar serviços.");

        } finally {
            setCarregando(false);
        }
    };

    useEffect(() => {
        carregarOrdens();
```

```
}, []);

if (carregando) return <p>Carregando...</p>;

if (erro) return <p>{erro}</p>;

return (
    <section>
        <h2>Ordens de serviço</h2>

        {ordens.map((ordem) => (
            <div key={ordem.id}>
                <p>
                    Serviço: {ordem.tipoServico}
                </p>

                <p>
                    Descrição: {ordem.descricao}
                </p>

                <p>
                    Veículo: {
                        ordem.veiculo?.modelo ||
                        ordem.modelo
                    }
                </p>

                <p>
                    Proprietário: {
                        ordem.veiculo?.proprietario?.nome ||
                        ordem.proprietarioNome
                    }
                </p>

                <p>Status: {ordem.status}</p>

                <p>
                    Valor: R$ {
                        Number(ordem.valor).toFixed(2)
```

```
                        }
                    </p>
                </div>
            ))}
        </section>
    );
};

export default OrdensServico;
```

A listagem apresenta informações relacionadas de três entidades: ordem de serviço, veículo e proprietário.

9.6 Dashboard simplificado

Arquivo `client/src/App.jsx`:

None

```
import { useState } from "react";

import Login from "./components/Login";
import Proprietarios from "./components/Proprietarios";
import Veiculos from "./components/Veiculos";
import OrdensServico from "./components/OrdensServico";

import "./App.css";

const App = () => {
    const [usuario, setUsuario] = useState(null);

    const sair = () => {
        sessionStorage.removeItem("token");
        setUsuario(null);
    };

    return (
        <main className="container">

            {!usuario ? (
```

```
                <Login
                    onLogin={(dados) => setUsuario(dados)}
                />
            ) : (
                <>
                    <header className="topo">
                        <div>
                            <h1>Gestão Automotiva</h1>

                            <p>
                                Bem-vindo(a), {usuario.nome}
                            </p>
                        </div>

                        <button onClick={sair}>
                            Sair
                        </button>
                    </header>

                    <div className="painel">
                        <Proprietarios />
                        <Veiculos />
                        <OrdensServico />
                    </div>
                </>
            )}

        </main>
    );
};

export default App;
```

O dashboard apresenta as principais listagens após a autenticação.

A implementação inicial não exige um sistema complexo de navegação. O objetivo é demonstrar a integração entre React, Axios, API e PostgreSQL.

9.7 CSS básico

Arquivo `client/src/App.css`:

```css
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f2f5f9;
    color: #243247;
}

.container {
    width: min(1100px, 92%);
    margin: 32px auto;
}

.topo {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    margin-bottom: 24px;
}

.painel {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 16px;
}

.painel section,
form {
    padding: 20px;
    background: white;
    border-radius: 10px;
    box-shadow: 0 2px 10px #00000012;
}
```

```
input {
    display: block;
    width: 100%;
    margin: 6px 0 14px;
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
}

button {
    padding: 10px 16px;
    border: 0;
    border-radius: 6px;
    background: #1769aa;
    color: white;
    cursor: pointer;
}

@media (max-width: 800px) {
    .painel {
        grid-template-columns: 1fr;
    }

    .topo {
        align-items: flex-start;
        flex-direction: column;
    }
}
```

10. Endpoints da API

Resumo das rotas esperadas:

Método	Endpoint	Função
POST	`/api/auth/login`	Realizar login.
GET	`/api/proprietarios`	Listar proprietários.

POST	`/api/proprietarios`	Cadastrar proprietário.
GET	`/api/veiculos`	Listar veículos e proprietários.
POST	`/api/veiculos`	Cadastrar veículo.
GET	`/api/ordens-servico`	Listar ordens de serviço.
POST	`/api/ordens-servico`	Criar ordem de serviço.
PATCH	`/api/ordens-servico/:id/status`	Atualizar status.

Todas as rotas, exceto login, devem exigir autenticação JWT.

11. Comparação das duas abordagens

Critério	SQL com pg	Sequelize
Consultas	Comandos SQL executados por `pool.query()`.	Métodos de Model como `findAll()`, `findOne()` e `create()`.
Relacionamentos	JOIN escrito diretamente em SQL.	Associações `hasMany()` / `belongsTo()` e opção `include`.
Estrutura do backend	Controllers trabalham com consultas e resultados.	Controllers trabalham com Models e instâncias.
Cadastro	INSERT com parâmetros.	Método `create()`.
Atualização	UPDATE com parâmetros.	Método `update()`.
Seed	INSERT SQL ou script JavaScript com pg.	Script JavaScript com `create()` ou `findOrCreate()`.
Principal vantagem didática	Prática direta de SQL, JOIN e parâmetros.	Prática de ORM, Models e associações.
Cuidados	Parâmetros, transações e manutenção das consultas.	Mapeamento, aliases, migrations e SQL gerado.

As duas alternativas são válidas.

O uso de ORM não elimina a necessidade de compreender SQL, chaves primárias, chaves estrangeiras, normalização e modelagem relacional.

12. Testes mínimos

Utilize Insomnia ou Postman para testar a API antes de desenvolver todas as telas.

12.1 Login

Enviar POST:

`http://localhost:3000/api/auth/login`

Corpo JSON:

```JSON
{
    "email": "admin@email.com",
    "senha": "123456"
}
```

Resultado esperado:

- HTTP 200.
- Retorno do token JWT.
- Retorno das informações básicas do usuário.

Senha incorreta deverá retornar HTTP 401.

12.2 Cadastrar proprietário

Enviar POST:

`http://localhost:3000/api/proprietarios`

```JSON
{
    "nome": "Carlos Silva",
    "email": "carlos@email.com",
    "telefone": "48999990000",
    "cpf": "11122233344"
}
```

Resultado esperado:

- HTTP 201.

- Retorno dos dados cadastrados.

12.3 Cadastrar veículo

Enviar POST:

`http://localhost:3000/api/veiculos`

```JSON
{
    "marca": "Volkswagen",
    "modelo": "Gol",
    "placa": "ABC1D23",
    "ano": 2020,
    "cor": "Branco",
    "proprietarioId": 1
}
```

Resultado esperado:

- HTTP 201.
- Veículo cadastrado e relacionado ao proprietário.

12.4 Criar ordem de serviço

Enviar POST:

`http://localhost:3000/api/ordens-servico`

```JSON
{
    "descricao": "Manutenção preventiva",
    "tipoServico": "Troca de óleo",
    "valor": 180.00,
    "veiculoId": 1
}
```

Resultado esperado:

- HTTP 201.
- Ordem de serviço criada.
- Status inicial `aberto`.

12.5 Atualizar status

Enviar PATCH:

`http://localhost:3000/api/ordens-servico/1/status`

```json
{
    "status": "em_andamento"
}
```

Resultado esperado:

- HTTP 200.
- Ordem de serviço com status atualizado.

12.6 Testes obrigatórios

- Login com credenciais válidas retorna token.
- Senha incorreta retorna HTTP 401.
- Acesso a uma rota protegida sem token retorna HTTP 401.
- Cadastro de proprietário persiste os dados e a listagem não expõe o CPF.
- Cadastro de veículo utiliza um proprietário existente.
- A placa não pode ser duplicada.
- Listagem de veículos apresenta os dados do proprietário relacionado.
- Ordem de serviço é vinculada ao veículo correto.
- Listagem de ordens apresenta veículo e proprietário.
- Atualização de status funciona.
- IDs inexistentes são rejeitados pela validação ou pelas chaves estrangeiras.
- Frontend apresenta dados, carregamento e mensagens de erro.
- Seed pode ser executado novamente sem duplicar os registros previstos pelo critério de busca.

13. Segurança e melhorias recomendadas

A primeira versão do sistema deverá manter a implementação simples, mas algumas melhorias são importantes:

- Validar os dados recebidos pela API, por exemplo com Zod, antes de gravar no banco.
- Implementar autorização por perfil (RBAC): autenticação identifica o usuário; autorização limita as ações permitidas.

- Adicionar rate limit, CORS restrito e tratamento centralizado de erros.
- Não retornar senhas ou CPF nas respostas desnecessárias.
- Cifrar o CPF com encriptação assincrona
- Validar a placa do veículo e impedir duplicidades.
- Impedir que ordens de serviço sejam criadas para veículos inexistentes.
- Restringir alterações de status conforme as regras da empresa.
- Evitar excluir veículos que possuam ordens de serviço relacionadas.
- Utilizar transações quando uma operação envolver várias gravações.
- Utilizar variáveis de ambiente para credenciais e segredos.

14. Checklist de conclusão

- Banco criado com quatro tabelas, chaves primárias e estrangeiras.
- Backend conecta ao PostgreSQL sem erros.
- Uma única estratégia de persistência está implementada de ponta a ponta.
- Models e associações implementados, caso tenha escolhido Sequelize.
- Seed insere pelo menos três registros por tabela.
- Senhas armazenadas utilizando bcrypt.
- Login e JWT estão funcionando.
- Rotas protegidas estão funcionando.
- Cadastro e listagem de proprietários estão funcionando.
- Cadastro e listagem de veículos estão funcionando.
- Ordens de serviço estão vinculadas aos veículos.
- Listagens apresentam os relacionamentos corretamente.
- Atualização de status está funcionando.
- Frontend React está consumindo a API por Axios.
- Dashboard apresenta as informações principais.
- Testes realizados com Insomnia ou Postman.
- Documentação e DER correspondem ao banco implementado.

Considerações finais

O projeto Gestão Automotiva permite trabalhar os mesmos conceitos de desenvolvimento backend e frontend em dois contextos empresariais diferentes.

Na oficina mecânica, as ordens de serviço representam manutenções, reparos e revisões.

No lava-car, representam lavagens, higienizações e outros serviços automotivos.

A estrutura do banco de dados, a autenticação, a API e o frontend permanecem os mesmos.

O principal objetivo é compreender como as entidades se relacionam, como o backend acessa o banco de dados e como o React consome as informações disponibilizadas pela API.

O aluno poderá escolher entre SQL com pg e Sequelize, mantendo os mesmos requisitos funcionais e resultados esperados.